Title Author Information

Sujal Pareek
Roll No. : 24f2007691
Email id : 24f2007691@ds.study.iitm.ac.in

Project Description

To design a role-based, session-authenticated web application for managing trekking events. The application supports three roles — *Administrator, Trek Staff, and User* — each with a dedicated dashboard. Admins create and manage treks and staff assignments, staff manage the treks assigned to them and view participants, and users browse, book, and cancel treks subject to slot availability.

Technologies Used

- **Flask**: for running the web application and handling routing
- **Flask-SQLAlchemy**: for connecting to and querying the database
- **Jinja2**: for server-side template rendering
- **Flask session**: for login state and role-based access control
- **SQLite**: as the underlying database

Database Schema Design

Users Table

Stores account details for admins, staff, and regular users.

- id (Integer, Primary Key, Not Null, Unique)
- name (String, Not Null)
- age (Integer, Not Null)
- email (String, Unique, Not Null)

- password (String, Not Null)
- mobile (String, Not Null)
- address (String, Not Null)
- role (String, Not Null) — admin / staff / user
- status (String, Default = "Active") — Active / Pending / Approved / Blacklisted
- created_at (DateTime, Default = current timestamp)

Trek Staff Table

Stores staff-specific profile details, linked one-to-one with a user account.

- id (Integer, Primary Key, Not Null, Unique)
- trek_staff_name (String, Not Null)
- phone (String, Not Null)
- experience (Integer, Not Null)
- address (String, Not Null)
- emergency_contact (String, Not Null)
- user_id (Integer, Foreign Key → users.id, Unique, Not Null)

Treks Table

Stores details of each trek listed by the administrator.

- id (Integer, Primary Key, Not Null, Unique)
- trek_name (String, Not Null)
- location (String, Not Null)
- difficulty (String, Not Null)
- duration (Integer, Not Null)
- available_slots (Integer, Not Null)
- start_date (Date)
- end_date (Date)
- status (String, Default = "Pending") — Pending / Open / Closed
- assigned_staff_id (Integer, Foreign Key → trek_staff.id, nullable)

Bookings Table

Stores each booking made by a user for a specific trek.

- id (Integer, Primary Key, Not Null, Unique)
- user_id (Integer, Foreign Key → users.id, Not Null)
- trek_id (Integer, Foreign Key → treks.id, Not Null)
- booking_date (DateTime, Default = current timestamp)
- booking_status (String, Default = "Booked") — Booked / Cancelled

- payment_status (String, Default = "Pending")

Architecture and File Structure

- **app.py**
- **models.py**
- **Creating_admin.py**
- **requirements.txt**
- **Read_Me.txt**
- **templates/**
    - **authentication/** *(index.html, register.html, register_staff.html)*
    - **admin/** *(dashboard.html, treks.html, trek_form.html, assign_staff.html, staff.html, users.html, bookings.html)*
    - **staff/** *(dashboard.html, trek_detail.html, participants.html)*
    - **user/** *(dashboard.html, treks.html, trek_detail.html, bookings.html, profile.html)*
- **instance/**
    - *trekking_app_database.sqlite3*

Features and Functionality

- **Role-Based Access Control**: Three distinct roles (Admin, Staff, User) authenticated via a single form.
- **Admin Seeding**: Default administrator account created via one-time setup script.
- **Registration Workflow**: Users register directly; staff registrations require admin approval.
- **Account Security**: Blacklisted accounts are strictly prohibited from logging in.
- **Administrative Capabilities**: Management of treks, staff assignments, user statuses, and global reporting.
- **Staff Operations**: Management of assigned treks, status updates, and participant viewing.
- **User Actions**: Browsing treks, booking management (based on slot availability), and profile updates.
- **Dynamic Inventory**: Automatic slot decrement/increment upon booking and cancellation.

AI Usage Declaration

AI Usage Percentage: *15-20 %*
I declare that the above percentage represents the extent to which AI-based tools were used during the preparation/development of this project. The final submission, code and report have been reviewed by me.

Video Reference

---

https://drive.google.com/file/d/1-u7ZKWCZUaMut9THQ81v7s59-WuGQ-Cy/view?usp=sharing

Conclusion

The Trekking Management Application demonstrates a complete role-based web application using **Flask , SQLAlchemy , SQLite and Jinja2**. It covers the core lifecycle of trekking management—from staff approval and trek creation to assignment, booking, participant tracking and trek status updates. The submitted implementation provides a functional foundation that can be extended with stronger security, validation, search, payments and analytics.
Final submission checklist: ZIP containing code + this 4-page report + 3–10 minute Google Drive demo video link + completed AI usage declaration.